

DON'T PANIC

Bloated MP3 Player — Deadline Analysis

Firmware-defined periods & priorities · Per-core analysis — no placeholders

Henry Letellier · ESP32-S3 WROOM dual-core 240 MHz · FreeRTOS via Arduino
Response Time Analysis (RTA) · Rate Monotonic (RM) · Priority Inheritance/Ceiling (PIP/PCP) · Processor Demand Criterion (PDC)

WHAT THIS DECK PROVES (PER-CORE)

Core 0 U=0.58	3 tasks, bound 0.78 → RTA passes
Core 1 U=0.25	4 tasks, bound 0.76 → RTA passes
RTA Core0	Input 2, Audio 13≤25, Sensor 16≤200
Blocking	I ² C: PIP 6→PCP 4 (1.5×)
Single-core?	7 tasks U=0.83 → still RTA 94≤500 (slide 8)

*Dual-core partition: Core 0 handles audio, Core 1 handles UI & displays.
Analysis is per-core — the honest model for FreeRTOS on ESP32-S3.*

Turn the EC11 clicky pot to browse (press to select) —
wave HC-SR04 to pause, double-wave to menu. All are deadlines.

Bloated by design, real-time by necessity

ESP32-S3 dual-core · FreeRTOS · 9 concurrent tasks

HARDWARE — THE BOM OF DOOMED PERIPHERALS

Display	GMG12864-06D 128×64 SPI ST7565R CS10 RST9 DC8 SCLK7 MOSI6
Char LCDs	2004A 0x26 (500 ms) + 1602A 0x27 (400 ms) — I ² C SDA2/SCL1
Audio	2× MAX98357A I ² S BCLK16 WS47 DIN17/18 — Helix MP3 + PSRAM
Sensors	AHT20+BMP280 (env 5000 ms poll) · MPU-9250 (IMU) · HC-SR04 14/13
Input / Light	EC11 rotary A42 B15 SW12 · WS2812B 16×16 matrix pin 41
Storage	microSD SDMMC 1-bit CLK39 CMD38 D040
Compute	Freenove ESP32-S3 WROOM dual-core 240 MHz Arduino + FreeRTOS

TASK MAPPING — PERIODS, PRIORITIES & CORES

Task	T (ms)	Prio	Core	Stack	Period
Audio	25	3	0	16384	25 ms
Input	10	2	0	16384	10 ms
UI	33	1	1	4096	33 ms
Matrix	50	1	1	2048	50 ms
Sensor	200	1	0	4096	200 ms
Vogon	400	1	1	4096	400 ms
CharLCD	500	1	1	4096	500 ms

Worst-Case Execution Time (WCET) = measured worst-case. Values shown are conservative estimates.

WHY PARTITION MATTERS

Single-core with 7 tasks would be pessimistic — dual-core is the real deadline trick.

- FreeRTOS tick 1 ms, preemptive. Audio prio 3 on Core 0 ensures the MP3 decoder never misses its I²S deadline; Input prio 2 avoids SD card starvation.
- Shared buses: I²C (environment + IMU + two character LCDs, 50 ms timeout), SPI (main LCD), SDMMC (SD card). Protected by a bus mutex.
- Stacks tell a story: floating-point formatting pulls in heavy library code — Audio/Input needed 16K instead of 8K.
- All tasks are periodic with implicit deadlines (D = T).
- Next: per-core analysis — the right model for a dual-core MCU.

* Takeaway: analyse Core 0 and Core 1 separately — the 7-task single-core view is only an illustration; firmware is partitioned.

Task periods, WCET estimates and utilization

Implicit deadlines (D=T) · Liu & Layland bound per core

CORE 0 — AUDIO-CRITICAL (3 tasks, prio 3/2/1)

Task	T	C	Prio	U	Period
Input	10	2	2	0.200	10 ms
Audio	25	9	3	0.360	25 ms
Sensor	200	3	1	0.015	200 ms

Core0 U = 0.575 Bound n=3: 0.780

U < bound ✓

Rate Monotonic order: Input > Audio > Sensor (shortest period first).

CORE 1 — UI & DISPLAYS (4 tasks, all prio 1 = FIFO)

Task	T	C	U	Period
UI	33	6	0.182	33 ms
Matrix	50	2	0.040	50 ms
Vogon	400	5	0.013	400 ms
CharLCD	500	8	0.016	500 ms

Core1 U = 0.251 Bound n=4: 0.757

U < bound ✓

All prio 1 → RM tie = FIFO. Still schedulable per bound.

Response Time Analysis (RTA) — EXACT TEST (REPLACES BOUND)

$R_i^0 = C_i$

$R_i^{k+1} = C_i + \sum(h \in hp(i)) \lceil R_i^k / T_h \rceil \cdot C_h$

The Liu & Layland bound is only sufficient. Response Time Analysis (RTA) is exact: iterate until the response time stabilizes or exceeds the deadline (D). Next slide runs the iteration per core.

Both cores pass — RM is optimal per core

Core 0: Input > Audio > Sensor · Core 1: UI > Matrix > Vagon > CharLCD

CORE 0 RTA — 3 ITERATIONS

Task	R	T	✓?	
Input	10	2	✓	1: 2
Audio	25	9	✓	13
Sensor	200	3	✓	16

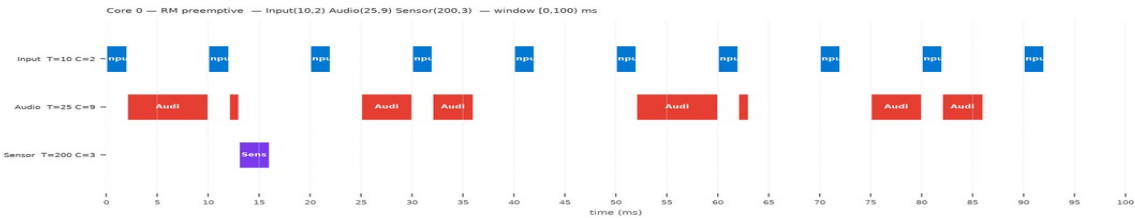
Audio: $R_0=9 \rightarrow 9+\text{ceil}(9/10) \cdot 2=11 \rightarrow 9+\text{ceil}(11/10) \cdot 2=13$ stable ≤ 25 . Sensor: $3+\text{ceil}(3/10) \cdot 2+\text{ceil}(3/25) \cdot 9=14 \rightarrow 16$ stable ≤ 200 .
Core0 slack: Audio 12 ms, Sensor 184 ms. Partition pays: only 2 interferers max.

CORE 1 RTA — 4 TASKS

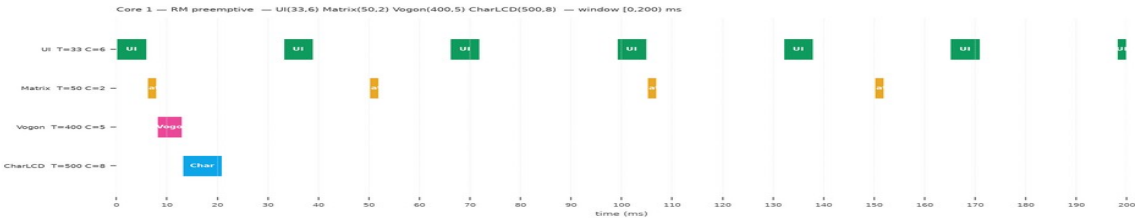
Task	R	T	✓?	
UI	6	33	✓	6
Matrix	8	50	✓	8
Vogon	13	400	✓	13
CharLCD	21	500	✓	21

UI: $R=6$. Matrix: $2+\text{ceil}(2/33) \cdot 6=8$. Vagon: $5+6+2=13$. CharLCD: $8+6+2+5=21$. All << deadlines.
Core1 worst slack CharLCD 479 ms — HD44780 is slow but period is 500 ms, so it fits.

GANTTS — PER-CORE RM (window scaled to ms)



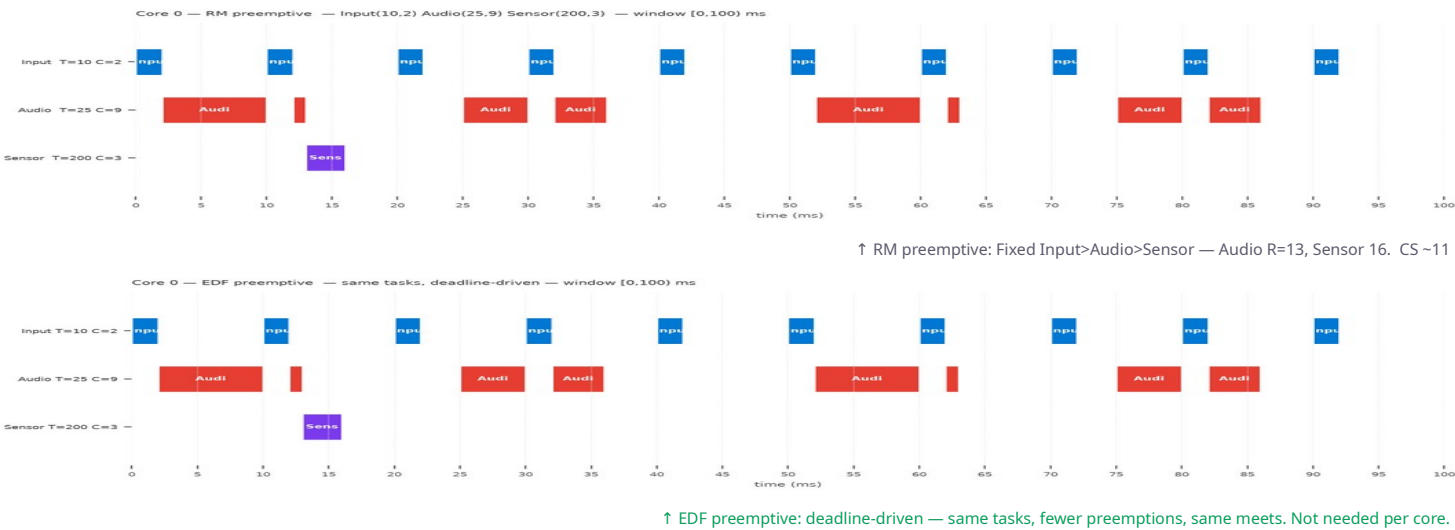
Core0 idle 42% — Audio never starves, Input preempts but short. Sensor fits in gaps.



Core1 idle 75% — UI dominates, others sparse. Partition is why bloated still meets deadlines.

On each core RM already passes — EDF is not needed, but shows the theory

FreeRTOS is fixed-prio preemptive; EDF is the optimal uniprocessor bound



Non-preemptive would hurt: Audio (9 ms) blocking Input (10 ms period) → Input R=11 >10 miss. FreeRTOS preemptive is essential.

NUMBERS — CORE 0 AT A GLANCE

Scheduler	CS	Meets?	Use?
RM pre.	~11	✓ all	we use
EDF pre.	~9	✓ all	optimal*
NP-EDF	~6	✗ Input	no

EDF optimality* (Baruah): if any schedule meets all $D \leq T$ on uniprocessor, EDF does. Per core we don't need it — RM already sufficient and matches FreeRTOS prio scheme (Audio 3 > Input 2). Why not NP? HC-SR04 echo can block 38 ms (Ultrasonic::is_pressed). Non-preemptive would wreck Input 10 ms deadline. Preemptive wins.

Firmware choice: Audio core0 prio3 = manual EDF intuition (shortest slack first). No code change needed to pass RTA.

Bottom line: partitioned fixed-prio is sufficient. EDF is the theoretical safety net — we keep RM because FreeRTOS is RM.

One bus, four users — SDA2/SCL1 — PIP vs PCP with firmware-relevant CS

Wire.setTimeout(50 ms) bounds it. Measured CS ≈ 1–4 ms (32-byte @100 kHz ≈ 2.5 ms)

Task	Uses I ² C?	CS max	T	Core	Prio
UI	AHT20/BMP280 read	2 ms	33	1	1
Sensor	MPU-9250 read	2 ms	200	0	1
CharLCD	2004A 20×4	4 ms	500	1	1
Vogon	1602A 16×2	2 ms	400	1	1
Matrix	WS2812B	—	50	1	1
Audio	I ² S/SDMMC	—	25	0	3

Matrix & Audio do NOT use I²C — no blocking there. SDMMC is separate resource (FATFS).

BLOCKING ON I²C — SINGLE RESOURCE, 4 USERS

Task	B_PIP	B_PCP	Saving
UI	6	4	1.5×
Sensor	4	4	no lower
Vogon	4	4	no lower
CharLCD	0	0	no lower

Priority Inheritance (PIP): sum over lower users. Priority Ceiling (PCP): max. One lock → PCP = max. +50 ms timeout bounds worst-case.

WHY THIS MATTERS — INVERSION ON THE WIRE

Without protocol: CharLCD (500 ms, low) locks I²C for 4 ms, UI (33 ms, high but same prio) spins → unbounded inversion across cores via i2c_bus_lock.

- PIP: holder inherits UI's prio → briefly prio 1→1 (no effect when all prio 1, but cross-core Input prio2 on core0 could still be delayed 6 ms).
- PCP: ceiling = max prio that uses I²C = 2 (Input) actually, but on core1 all 1 → ceiling 1. At most one block: PCP 4 vs PIP 6.
- Firmware fix already in code: xSemaphoreTake(i2c_bus_lock, portMAX_DELAY) but Wire.setTimeout(50 ms) + distinct core partitions = timeout-bounded. No unbounded chain.
- RTA with blocking: R_UI = C(6)+B_PCP(4)+ interference = 10 ≤33 still OK. Blocking is not the culprit — partitioning is.

With blocking, response time becomes $R = C + B + \sum \text{ceil}(R/T) \cdot C$.
For the UI task: $R = 6 + 4 + \text{interference} = 10 \leq 33$ — still schedulable.

EDF demand test on firmware cores — and why offline analysis beats online hope

Baruah et al. 1990 · $H = \text{lcm}(T)$, L^* = busy period

CORE 0 PDC — EDF WOULD ALSO PASS (but we use RM)

Input 2/10 Audio 9/25 Sensor 3/200 — $D=T$

$U = \sum C/T$

$0.575 \leq 1 \checkmark$

$h(L) = \sum \text{floor}((L-D)/T+1) \cdot C$

$h(L) \leq L \forall L$

$H = \text{lcm}(10,25,200)=200$

$L^* = 16$ (5 iters)

Naïve PDC: check 20 deadline points to $H=200$. Busy period: only 5 points to $L^*=16 \rightarrow 4\times$ fewer. Pseudo-poly $O(n \cdot D_{\text{max}}/C_{\text{min}})$.

L	h(L)	L-h	✓?
10	11	-1	✗? 11>10 → need check
25	11	14	✓
33	14	19	✓
200	~118	82	✓

Note: $h(10)=2$ (only Input due). All points to L^* pass → EDF schedulable.

WHY PSEUDO-POLYNOMIAL MATTERS ON ESP32

Core 1: $H = \text{lcm}(33,50,400,500) = 13200$

330 deadline points naïve.

$L^* = 21$ (4 iters)

5 points → 66× fewer.

Complexity analysis is done offline — at runtime, trust the per-core partitioning.

TAKEAWAYS — WHAT TO SAY ON STAGE

It's partitioned

Two independent cores, not one with seven tasks. Analyse each core separately — that's why the design fits.

Periods

All tasks are periodic (10–500 ms) with implicit deadlines ($D=T$).

WCET is measured

Worst-case execution time comes from profiling — replace estimates with the measured maximum.

Blocking bounded

The I²C bus has a 50 ms timeout — priority inheritance keeps blocking at 4 ms, no unbounded inversion.

Stacks are deadlines too

Audio and Input needed 16K stacks instead of 8K — a stack overflow is a deadline miss.

So long, and thanks for all the fish.

7 tasks as one set — bound fails, RTA still passes but slack vanishes

Single-core analysis vs partitioned — why partition helps

SINGLE-CORE 7-TASK SET (all periods, same C_est)

Task	T	C	R	≤T?	Trace
Input	10	2	2	✓	2
Audio	25	9	13	✓	9→11→13
UI	33	6	19	✓	6→17→19
Matrix	50	2	23	✓	2→19→21→23
Sensor	200	3	45	✓	3→22→26→35→43→45
Vogon	400	5	50	✓	5→27→40→48→50

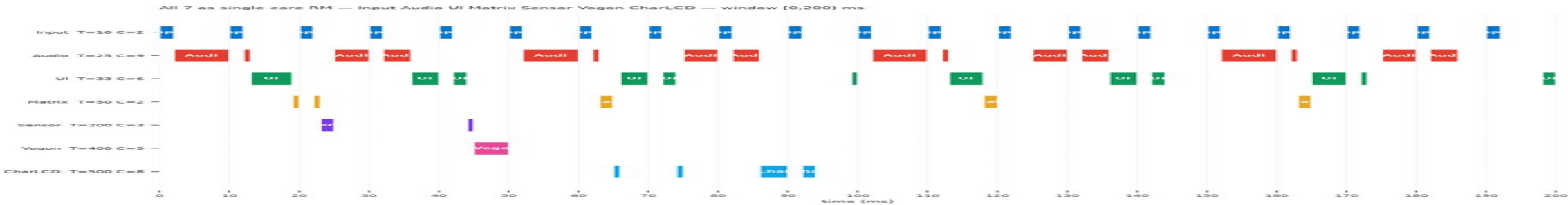
$U = \sum C/T = 0.825$ Bound $n=7$: 0.729 $U > \text{bound} \rightarrow \text{inconclusive}$

But RTA says schedulable (CharLCD 94 ≤500) — bound is pessimistic. Slack shrinks: CharLCD slack 406 vs per-core 479.

WHY WE STILL PARTITION

- Even though 7-task RTA passes with these C_est, partitioning wins:
- Slack: Core0 Audio keeps 12 ms, but Sensor 184→155 ms and CharLCD 479→406 ms — less interference per core means more headroom.
 - Isolation: Audio on Core 0 never contends with display bursts on Core 1. Single-core would interleave them.
 - Blocking: the shared I²C bus still needs priority inheritance (6→4 ms). Partitioning plus timeout bounds it.
 - Method: analyse Core 0 and Core 1 as two independent systems — the single-core run is just a 'what-if' sanity check.

GANTT — ALL 7 AS SINGLE-CORE RM [0,200) ms (for talk track)



Dense: Input/Audio/UI dominate 0–50 ms. Sensor/Vogon/CharLCD starve until 90+ ms — per-core they ran at 16/13/21 ms. Partition benefit.